



Home

Principles of Programming Languages

Page contents

- [2024-2025](#)
- [Curriculum](#)
- Older Material (not maintained): [2023](#), [2022](#)
- [Motivation](#)
- [Useful links and Bibliography](#)
- [Lecture notes](#)
- Course description
 - [PDF document in Romanian](#)
 - [PDF document in English](#)
- [Schedule](#)
- Email: andrei.arusoaie@uaic.ro

[\[Top\]](#)

Contents

- Introduction to Programming languages. The main programming paradigms.
- Coq. Inductive definitions. Inductively defined functions.
- Higher-Order Functions. Induction principles. Proofs.
- Abstract syntax.
- Abstract machines. Defining interpreters for imperative languages.
- Operational semantics: Big-step and Small-step
- Type systems.
- Compilation. Proving the soundness of a compiler.
- Paradigms: object-oriented programming, functional programming, logic programming

Motivation & Description

1. How often do you need to learn/design a new programming language?

- Nowadays, pretty often. There is a plethora of new languages, other than the classical ones (e.g., C, Java, C++), that have been invented in order to fulfill special needs. For instance, domain specific markup languages (Markdown, ColdFusion), functional programming languages (Erlang), compiler writing languages used within metacompilers, etc. The question is how easy is to learn or design such languages?

Description. This course attempts to answer exactly to this challenge, by covering the main **concepts** that a language designer needs to know. Moreover, the course is also useful for programmers, since it facilitates switching from one programming language to another.

2. Have you ever wondered why sometimes programs behave in a completely unexpected way?

- For example, consider the following program:

```
:~$ cat file.c
int main(void) {
    int x = 0;
    return (x = 1) + (x = 2);
}
```

One would expect that the value returned by this program is 3, since $x = 1$ evaluates to 1 and $x = 2$ evaluates to 2. However, the execution of the program does not exhibit the expected result:

```
:~$ gcc file.c
:~$ ./a.out ; echo $?
4
```

This is because in C, the evaluation order of the operands of $+$ is not fixed. So, there is some **undefinedness** involved in this C program. Compilers take advantage of this and choose whatever behavior they want for their programs. In `gcc`, the above program is translated into:

```
int x = 0;
x = 1;
x = 2;
return x + x;
```

Now, obviously, the returned value (i.e., 4) makes sense. However, this is not what a programmer would expect. This strange behavior of the `gcc` compiler is due to the **undefinedness** of this program (note: in the C standard, the *semantics* of that program is not defined).

Description. In this course, we address the problem of designing new programming languages by defining their semantics formally. This enables a series of analyses that can be done over programs, including detection of undefined behavior.

3. How important is formal semantics of a programming languages?

- Let us consider another tricky program written in C:

```
:~$ cat out-of-lifetime.c
int *foo(int x) {
    int z = x;
    int *y = &z;
    return y;
}
int main() {
```

```
int *x = foo(0);
foo(255);
return *x;
}
```

There is nasty thing happening here: `foo` returns a pointer to a local variable, which is a pointer in the stack. In `main`, we call `foo` twice with different arguments: first we call it with 0 (and we keep a pointer in the stack `*x`), and then we call it with 255. What do you think is the output of this? Well... let's see what `gcc` says:

```
:~$ gcc --version
gcc (Ubuntu 4.8.4-2ubuntu1~14.04.1) 4.8.4
Copyright (C) 2013 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
:~$
:~$ gcc out-of-lifetime.c
:~$ ./a.out ; echo $?
255
```

This seems right, since for the two calls, two corresponding frames were switched in the stack, and `*x` points to 255. Let us go further and use some `gcc` optimisation options, like `-O3`:

```
:~$ gcc -O3 out-of-lifetime.c
:~$ ./a.out ; echo $?
0
```

So, we have used the same *program*, compiled with the *same* compiler and we got different results!

In this course, we go through the process of defining both the syntax and the semantics of programming languages. We will see how tricky aspects can be captured by the semantics of a programming language and we will try out various tools proposed by the research community.

[\[Top\]](#)

Useful links and Bibliography

[1] *Practical Foundations of Programming Languages*, Robert Harper; Cambridge University Press, 2016. [Online book](#)

[2] *Software Foundations - Vol 2*, Benjamin C. Pierce, Arthur Azevedo de Amorim, Chris Casinghino, Marco Gaboardi, Michael Greenberg, Cătălin Hrițcu, Vilhelm Sjöberg, Andrew Tolmach, Brent Yorgey. [Online book](#)

[3] *Programming Languages: Principles and Paradigms*, Maurizio Gabbriellini, Simone Martini; Springer-Verlag London Limited, 2010. [Springer link](#).

[4] *The formal semantics of Programming Languages – An Introduction*, Glynn Winskell, 1993. The MIT Press, Cambridge, Massachusetts, ISBN 978-0-262-23169-5. [Online book](#).

[\[Top\]](#)

Last updated: 01/09/2025 09:05:09 - Copyright © Andrei Arusoaie

